

This is one of the coolest math tricks in programming! It works because Rock, Paper, Scissors is inherently **cyclical** (a loop where everything beats one thing and loses to another).

By assigning numbers to the choices, we can use the **modulo operator (%)**—which finds the remainder of a division—to calculate the winner instantly.

Let's break down exactly why it works.

1. The Setup (The Wheel of Superiority)

You assigned the numbers like this:

- 0 = Rock
- 1 = Paper
- 2 = Scissors

Because of the rules, each choice is strictly beaten by the number right *after* it:

- Rock (0) is beaten by Paper (1) 🖐️ $1 - 0 = 1$
- Paper (1) is beaten by Scissors (2) 🖐️ $2 - 1 = 1$
- Scissors (2) is beaten by Rock (0) 🖐️ *This resets the loop.*

Notice how in the first two standard matchups, **the winning index is exactly 1 greater than the losing index.**

2. Why the Formula Works for Standard Wins

Let's plug your winning scenarios into the formula: $(\text{user_input} - \text{computer_input}) \% 3$

Scenario A: You choose Paper (1), Computer chooses Rock (0) — YOU WIN

- Math: $1 - 0 = 1$
- Divide 1 by 3: $1 \div 3 = 0$ with a **remainder of 1.**
- Result: $1 == 1$ ✅ **True**

Scenario B: You choose Scissors (2), Computer chooses Paper (1) — YOU WIN

- Math: $2 - 1 = 1$
- Divide 1 by 3: $1 \div 3 = 0$ with a **remainder of 1.**
- Result: $1 == 1$ ✅ **True**

3. The Magic of Modulo on Negative Numbers (The Reset)

What happens when you choose Rock (0) and the Computer chooses Scissors (2)? You still win, but the math looks broken at first:

- Math: $0 - 2 = -2$

Here is where Python's modulo operator works its magic. In mathematics and Python, when you take the modulo of a negative number, it wraps around the clock.

To find $-2 \bmod 3$, Python thinks: "*What is the next multiple of 3 higher than -2?*" That multiple is 3.

It then calculates the distance: $\$3 + (-2) = 1\$$.

- Therefore, $-2 \% 3$ in Python evaluates to 1.
- Result: $1 == 1$  **True**

 **Think of it like a clock:** If a clock says 12:00 (0) and you go backward 2 hours (-2), you land on 10:00. In a 3-hour game world, going backward 2 steps from 0 wraps you right around to position 1!

Summary of All Outcomes

Because of this cyclical math, the equation $(\text{user_input} - \text{computer_input}) \% 3$ will *always* yield one of three numbers:

If the remainder is...	What it means
0	It's a Tie (e.g., $\$1 - 1 = 0\$$; $\$0 \bmod 3 = 0\$$)
1	You Win (The user is exactly 1 step ahead on the wheel)
2	You Lose (The computer is 1 step ahead, which is the same as being 2 steps behind)

Instead of writing out 6 different conditions, this single line of math covers every single winning scenario perfectly!